Proceso de pruebas

Asesoría: Planeación y diseño de pruebas.

Prof. Ulises Vélez Saldaña

ESCOM

15 de octubre de 2026











1. Procesos de pruebas
   - Las pruebas dentro del proyecto
   - Control de pruebas
   - Plan de pruebas
   - Análisis y diseño de pruebas
   - Implementación y ejecución de pruebas
   - Evaluación y generación de informes
   - Actividades de cierre
2. Modelos de desarrollo
3. Niveles de prueba



El proceso de pruebas

Dependiendo del enfoque seleccionado, el proceso de pruebas tendrá lugar en diferentes puntos del proceso de desarrollo:

- Las pruebas constituyen un proceso en sí mismas.
- El proceso de pruebas está determinado por las siguientes fases:
  - Planificación de pruebas y Control
  - Análisis de pruebas y diseño de pruebas
  - Implementación de pruebas y ejecución de pruebas
  - Evaluación de criterio de finalización de pruebas y generación de informes de pruebas.
  - Actividades de cierre de pruebas.



Proceso de pruebas a lo largo del desarrollo de software

- El proceso de pruebas es más que la ejecución de pruebas.
- Incluye superposición y vuelta atrás (backtracking).
- Cada fase del proceso de pruebas tiene lugar de forma concurrente con las fases del proceso de software.

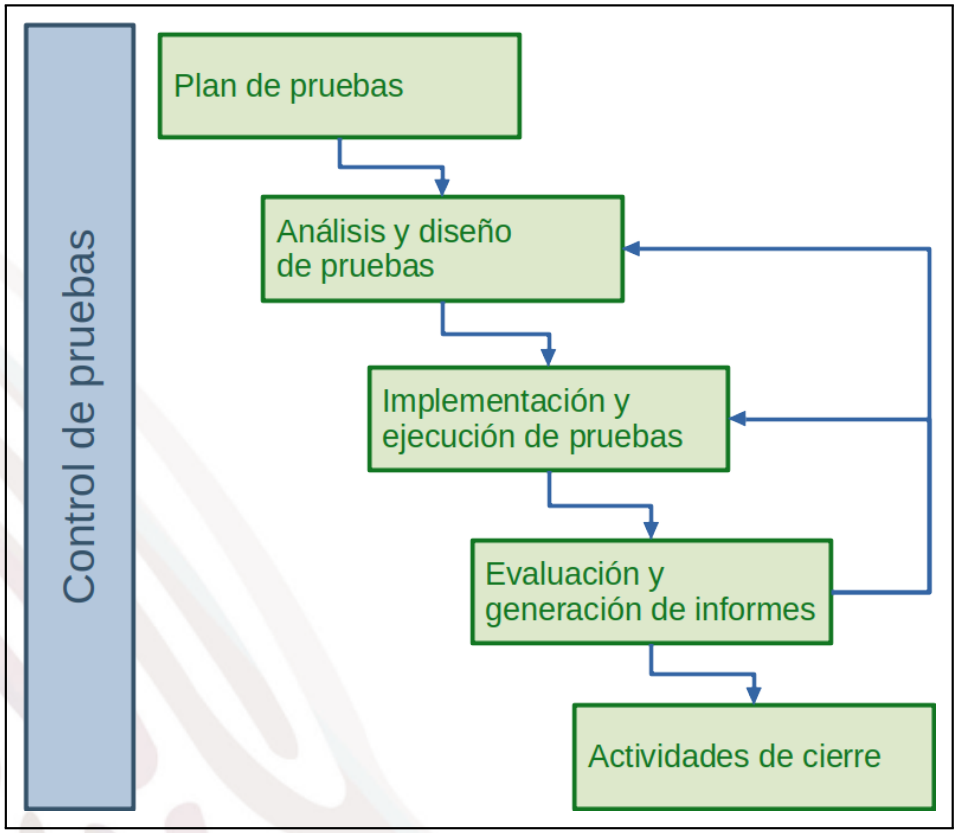
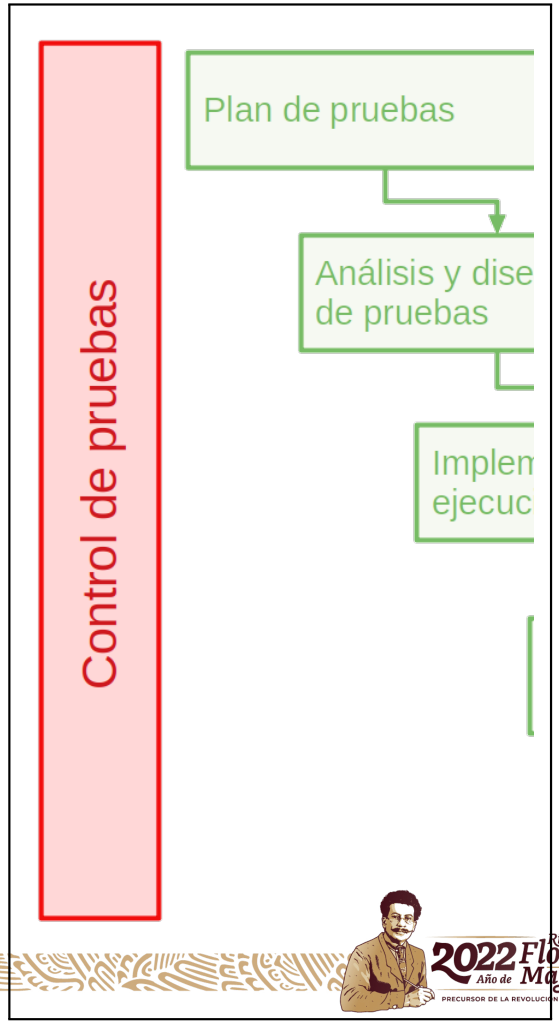


Figura: Proceso de pruebas



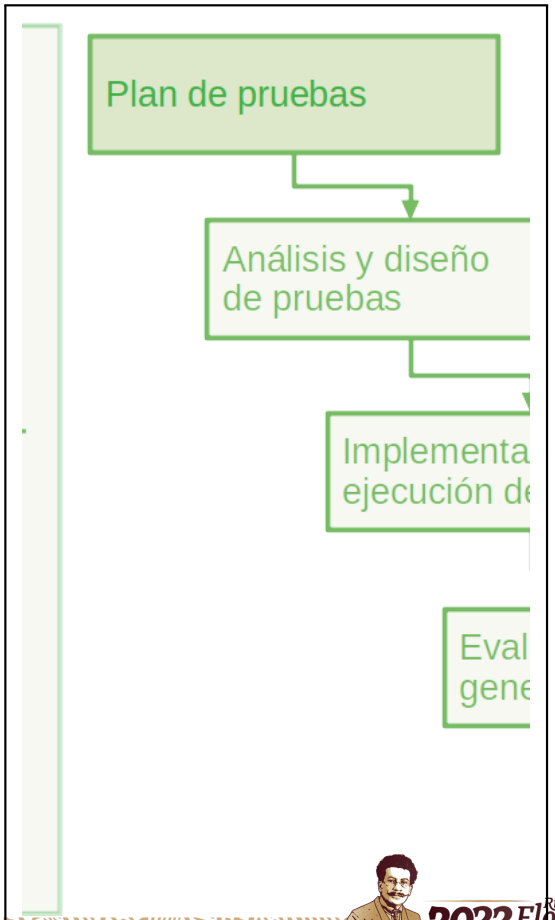
Control: Tareas principales

- Actividad continua que influye en la planeación. El Plan de pruebas maestro puede ser modificado en función de la información adquirida a partir del control de pruebas.
- Compara el progreso logrado respecto al plan de pruebas e inicia las actividades necesarias consecuentes.
- Se miden y analizan resultados.
  - La evolución de las pruebas.
  - La cobertura de las pruebas.
  - El cumplimiento de los criterios de salida.
- Se inician medidas correctivas.
- Se preparan y toman medidas correctivas.



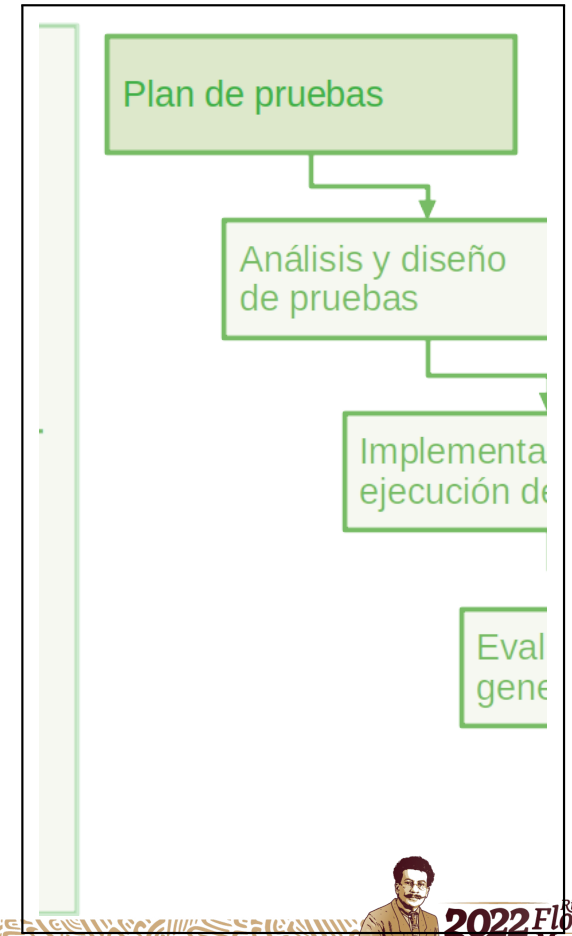
Plan de pruebas: Tareas principales

- Determinar el alcance y riesgos
- Identificar los objetivos de las pruebas y los criterios de salida de las pruebas.
- Determinar el enfoque de las pruebas, técnicas de pruebas, cobertura de las pruebas y el equipo de pruebas.
- Implementar el método de pruebas y las estrategias de pruebas. Se debe planear el periodo de tiempo para el desarrollo de las actividaeds de pruebas.
- Adquirir y programar los recursos requeridos por las pruebas: personal, entorno de pruebas, presupuesto.



Plan de pruebas: Artefactos principales

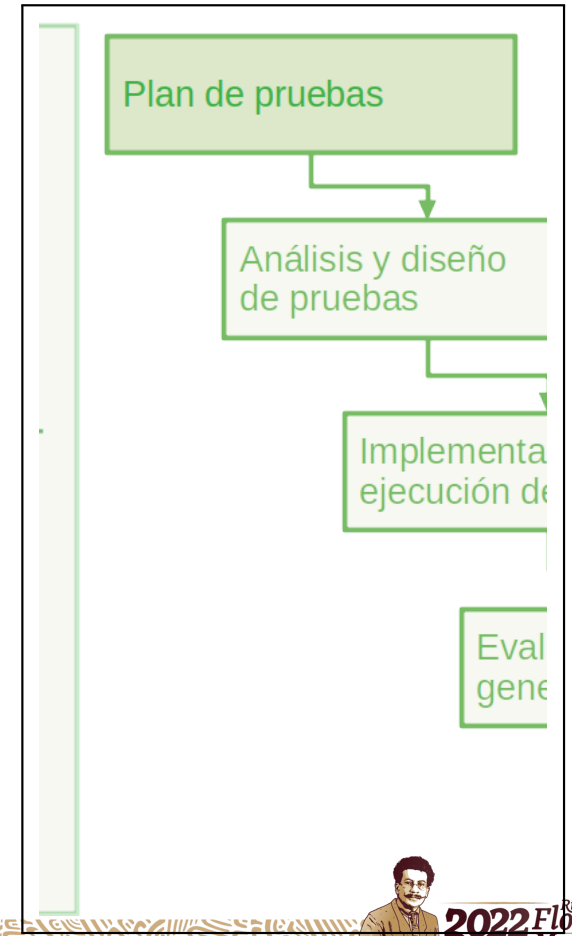
- Plan de pruebas maestro Documento en el que se describe el alcance, recursos y calendario de las actividades de pruebas. Este documento incluye pero no está limitado a:
  - Elementos de prueba.
  - Características que serán probadas.
  - Recursos.
  - Planificación de contingencias.
- Estrategia de pruebas Descripción de alto nivel de pruebas a llevar a cabo y las pruebas asociadas a ellos para una organización o programa.



Plan de pruebas: Artefactos principales

Enfoque de pruebas Normalmente incluye:

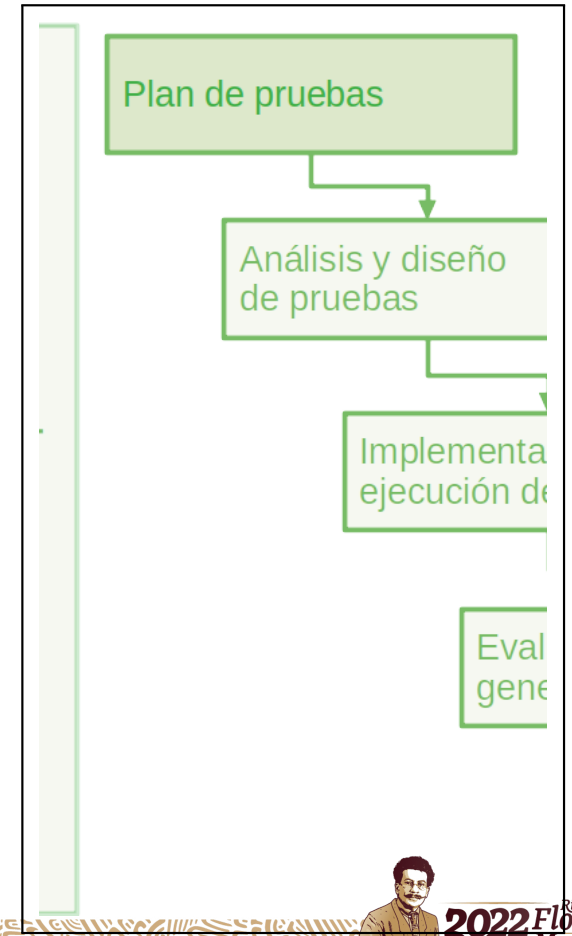
- Las decisiones tomadas con el objeto de lograr los objetivos del proyecto
- El análisis de riesgos.
- Puntos de inicio respecto del proceso de pruebas.
- Técnicas de diseño de pruebas a aplicar.
- Criterios de salida.
- Tipos de pruebas a ejecutar.



Plan de pruebas: Artefactos principales

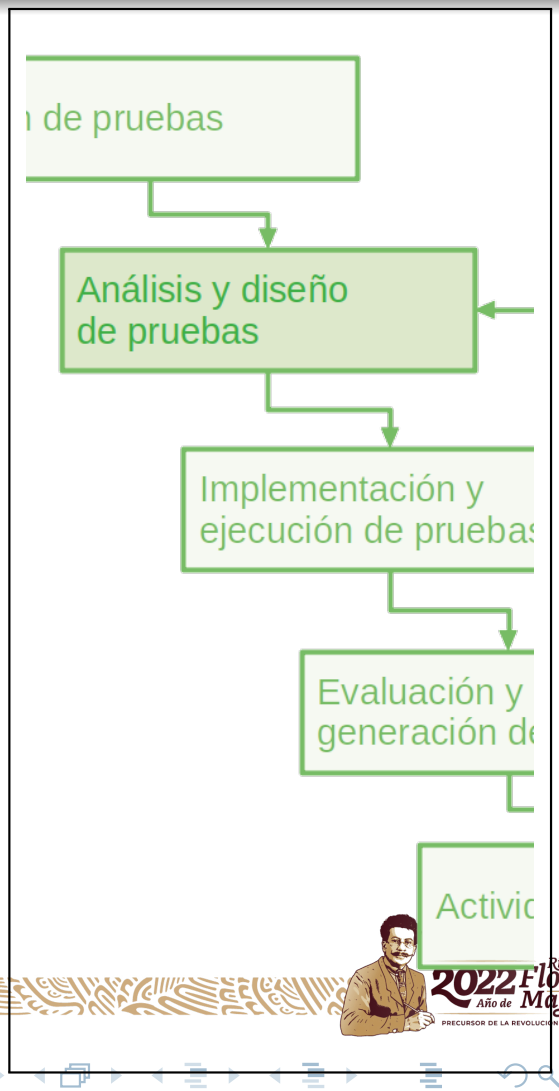
Criterios de salida

- Conjunto de condiciones genéricas y acordadas con los involucrados, para que un proceso sea considerado formalmente concluido.
- El propósito de los criterios de salida es evitar que una tarea se considere concluida habiendo partes destacadas de la misma sin completar.
- Los criterios de salida son utilizados como referencia para la elaboración de informes y para planificar cuando se deben finalizar las pruebas.
- Lo anterior debe ser realizado para cada nivel de pruebas.



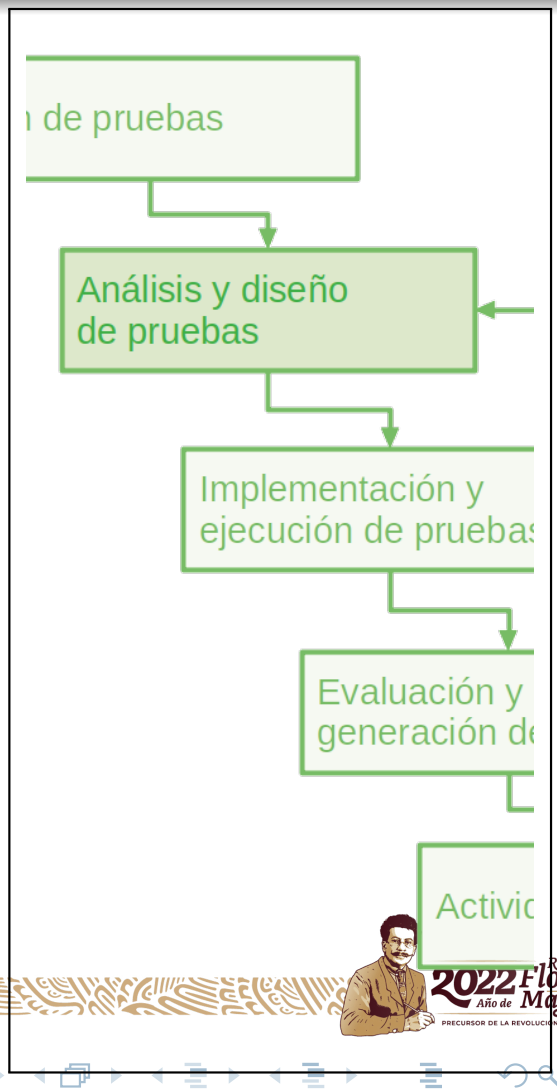
Análisis y diseño: Tareas principales

- Revisar las bases de pruebas (requisitos, arquitectura, diseño, interfaces, etc.)
- Analizar la "testabilidad" Evaluar la viabilidad de construir los casos de prueba con base en la base de pruebas.
- Identificar y priorizar condiciones de prueba (test conditions) en función de:
  - Análisis de los elementos de prueba.
  - Especificación de pruebas.
  - Comportamiento y estructura del software



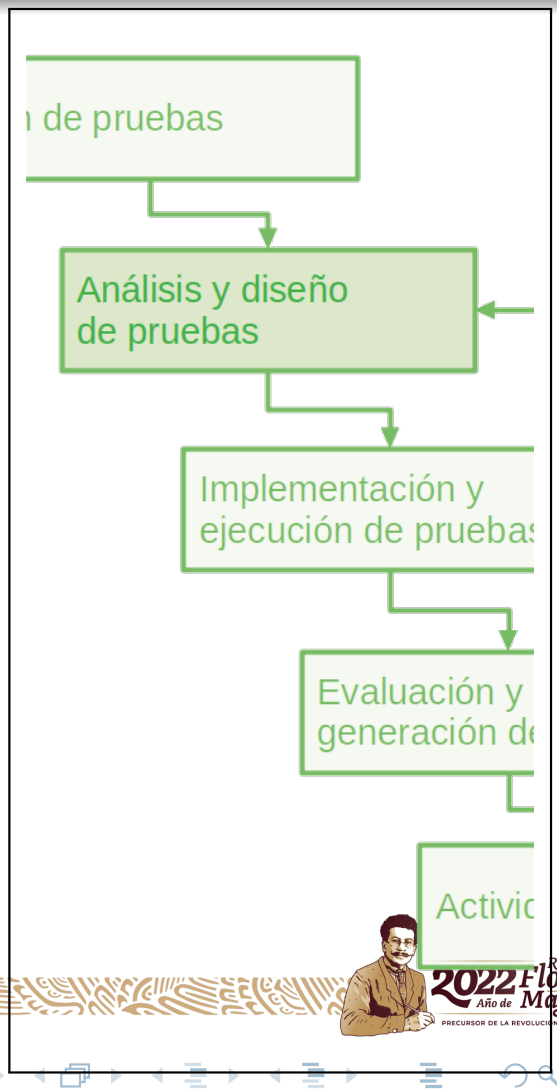
Análisis y diseño: Tareas principales

- Diseñar pruebas y casos de prueba
  - Crear y priorizar casos de prueba lógicos o de alto nivel.
  - Los casos de prueba positivos dan muestra de la funcionalidad.
  - Los casos de prueba negativos comprueban situaciones en las que hay tratamiento de errores.
- Identificar condiciones de prueba específicas y los datos de prueba necesarios
  - Evaluar la disponibilidad de datos de prueba y/o la viabilidad de la generación de datos de prueba.



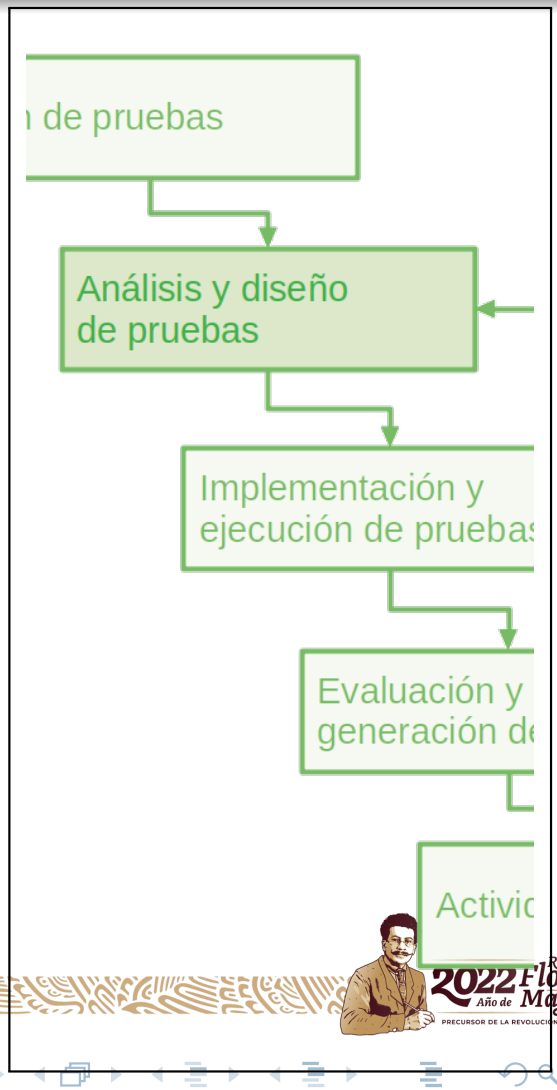
Análisis y diseño: Tareas principales

- Diseñar el entorno de prueba (test environment)
- Este puede ser exclusivo lo cual garantiza su disponibilidad o se pueden definir ventanas temporales para las pruebas (test windowed).
- Definir la operación del entorno de pruebas, incluyendo la administración de usuarios.
- Cargar conjuntos de datos y parámetros del sistema.
- Conectar al entorno de pruebas con los sistemas adyacentes (sand-box)



Análisis y diseño: Tareas principales

- Probar la infraestructura y herramientas de pruebas si fuera necesario.
  - Procesos, procedimientos y responsabilidades.
  - Seleccionar, proveedor, instalar y operar herramientas de prueba.
- Crear trazabilidad bidireccional.
  - Entre las bases de las pruebas y los casos de prueba.



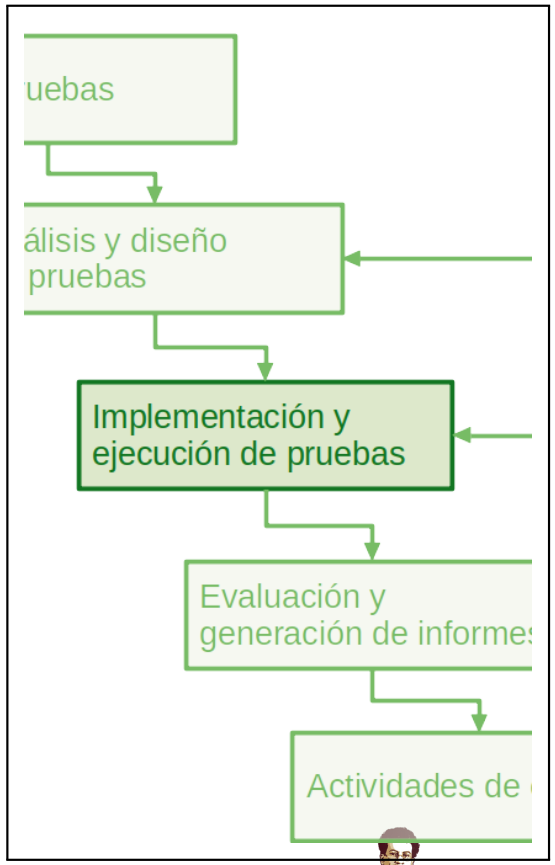
Análisis y diseño: Artefactos principales

- Datos de prueba Datos que existen en el sistema antes de que una prueba sea ejecutada y que afecta o es afectado por el componente o sistema sujeto a prueba.
- Datos de entrada Variable que es leída por un componente.
- Cobertura de pruebas Grado en el que un elemento especificado ha sido practicado por un juego de pruebas. Utilizado con mayor frecuencia en pruebas de caja blanca.
- Oráculo de pruebas Fuente que permite determinar los resultados esperados de un software sujeto a pruebas: comparativas o benchmarks, resultados de pruebas previas, manuales de usuario o conocimiento especializado.



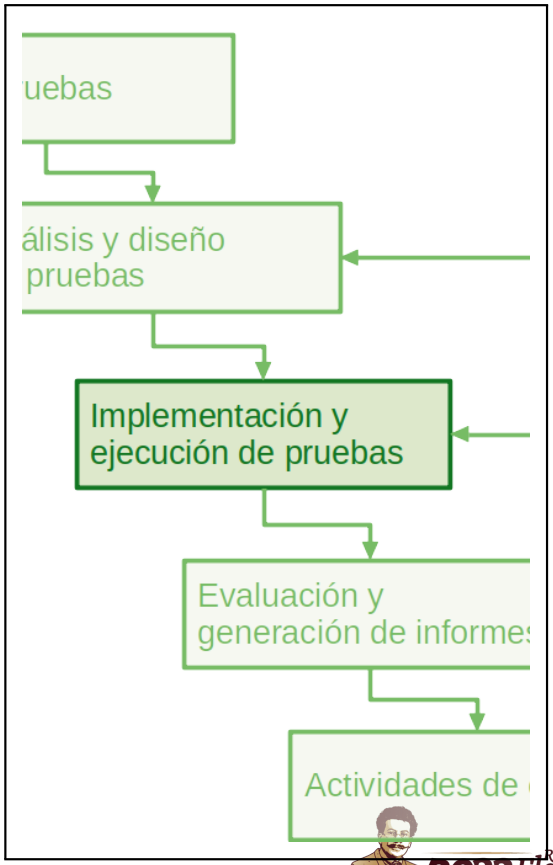
Implementación y ejecución de pruebas: Tareas principales

- Finalizar, implementar y priorizar casos de prueba.
- Desarrollar y priorizar procedimientos de pruebas:
  - Crear datos de pruebas.
  - Preparar arneses de pruebas.
  - Redactar guiones de pruebas automatizados.
  - Crear juegos de pruebas de los procedimientos para una ejecución de pruebas eficiente.
- Verificar el entorno de prueba.



Implementación y ejecución de pruebas: Tareas principales

- Verificar y actualizar trazabilidad.
- Ejecutar pruebas.
- Registrar resultados de pruebas y análisis
  - Registrar los identificadores y versiones de las herramientas y el probado (testware).
  - Comparar resultados reales con los resultados esperados.
- Informar y analizar incidencias con el objeto de establecer su causa: Código, producto de soporte de pruebas, documento o ejecución.
- Repetición de pruebas (re-testing).
- Ejecutar Pruebas de regresión (regression test):
  - Asegura que los cambios no han revelado otros defectos o introducido nuevos.



Implementación y ejecución de pruebas: Artefactos

- Juego de pruebas (test-suit)/ Secuencia de pruebas (test-sequence) Conjunto de casos de prueba para un componente o sistema en pruebas, donde la postcondición de una prueba es utilizada como precondición de la siguiente.
- Especificación de procedimiento de pruebas (test procedure specification) Documento que especifica la secuencia de acciones para la ejecución de una prueba. También conocido como script de pruebas o script de pruebas manual.
- Ejecución de pruebas (test execution) Proceso de practicar una prueba produciendo resultados reales.



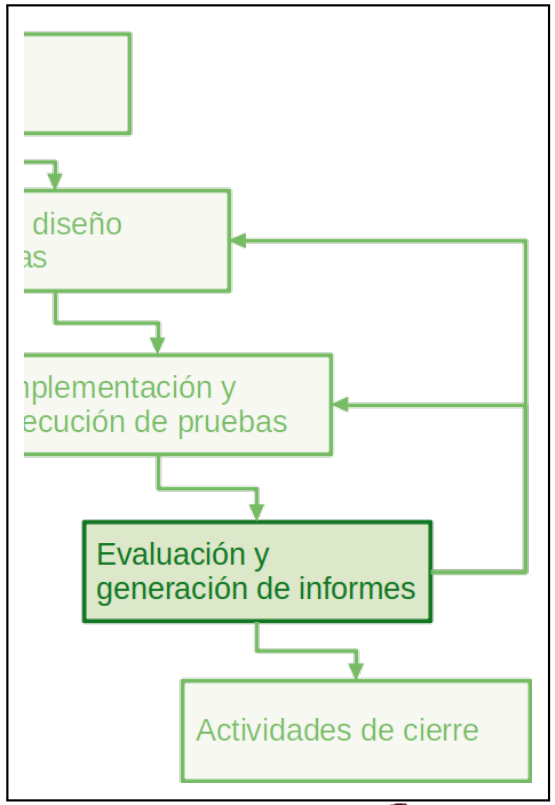
Implementación y ejecución de pruebas: Conceptos

- Registro de pruebas (test-log, test protocol, test report) Registro cronológico de los detalles relevantes respecto a la ejecución de pruebas: ¿Cuándo se desarrollaron las pruebas?, ¿Qué resultados fueron generados?
- Pruebas de regresión (regression tests) Pruebas de un programa previamente probado que ha sufrido modificaciones para asegurarse que no se han introducido o descubierto defectos en áreas del software que no han sido modificadas como resultado de los cambios realizados. Se realiza cuando el software o su entorno han sido modificados.
- Repeticion de pruebas (re-testing) Repetición de una prueba tras la corrección de un defecto con el objeto de confirmar que el defecto ha sido eliminado con éxito.



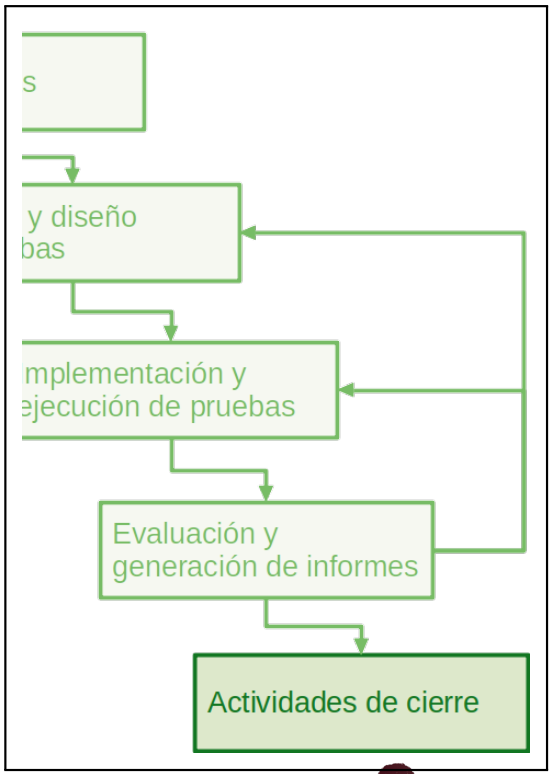
Evaluación y generación de informes: Tareas principales

- Evaluar la ejecución de pruebas con respecto a objetivos definidos (por ejemplo, criterios de salida).
- Evaluar los registros de prueba (resumen de las actividades de pruebas, resultados de prueba, comunicar criterio de salida).
- Proporcionar información con el objeto de dar lugar a la decisión de llevar a cabo pruebas adicionales.



Actividades de cierre: Tareas principales

- Recopilar datos de las actividades del proceso de pruebas finalizadas.
- Cerrar Informe de incidencias o generación de solicitudes de cambio para cualquier punto que permaneciera abierto.
- Comprobar que los Entregables planeados han sido entregados y aprobados.
- Documentar la aceptación del sistema.
- Finalizar y archivar los productos de soporte de pruebas (text-ware).
- Analizar las lecciones aprendidas para futuros proyectos.
- Utilizar la información recopilada para mejora la madurez del proceso de pruebas.



Resumen

El proceso de pruebas se puede dividir en diferentes fases.

- Planificación de pruebas abarca las actividades de definición de enfoque y planificación de recursos.
- Diseño de pruebas Abarca el diseño de casos de prueba y sus resultados esperados.
- Ejecución de pruebas Abarca la definición de datos de prueba, la ejecución de pruebas y la comparación de resultados.
- Evaluacion de pruebas Abarca la evaluación del criterio de salida y el registro de los resultados de pruebas en forma escrita.
- Control de pruebas Consiste en el control de las actividades que cubren todas las fases del proceso de pruebas.



1 Procesos de pruebas

2 Modelos de desarrollo
- Modelos-V general
- Niveles de pruebas
- Verificación vs Validación
- Modelo de desarrollo W
- Modelos iterativos

3 Niveles de prueba



El modelo de desarrollo en V

- El modelo de desarrollo en V se parece mucho a una cascada pero colocando énfasis en las pruebas.



El modelo de desarrollo en V

- El modelo de desarrollo en V se parece mucho a una cascada pero colocando énfasis en las pruebas.
- Este modelo de pruebas se usa como base para explicar que es un Nivel de pruebas.



El modelo de desarrollo en V

- El modelo de desarrollo en V se parece mucho a una cascada pero colocando énfasis en las pruebas.
- Este modelo de pruebas se usa como base para explicar que es un Nivel de pruebas.
- Las pruebas de un sistema se recomiendan en todos los niveles del sistema.



El modelo de desarrollo en V

- El modelo de desarrollo en V se parece mucho a una cascada pero colocando énfasis en las pruebas.
- Este modelo de pruebas se usa como base para explicar que es un Nivel de pruebas.
- Las pruebas de un sistema se recomiendan en todos los niveles del sistema.
- Un nivel de pruebas hace énfasis en que elementos se están probando y bajo que perspectiva.

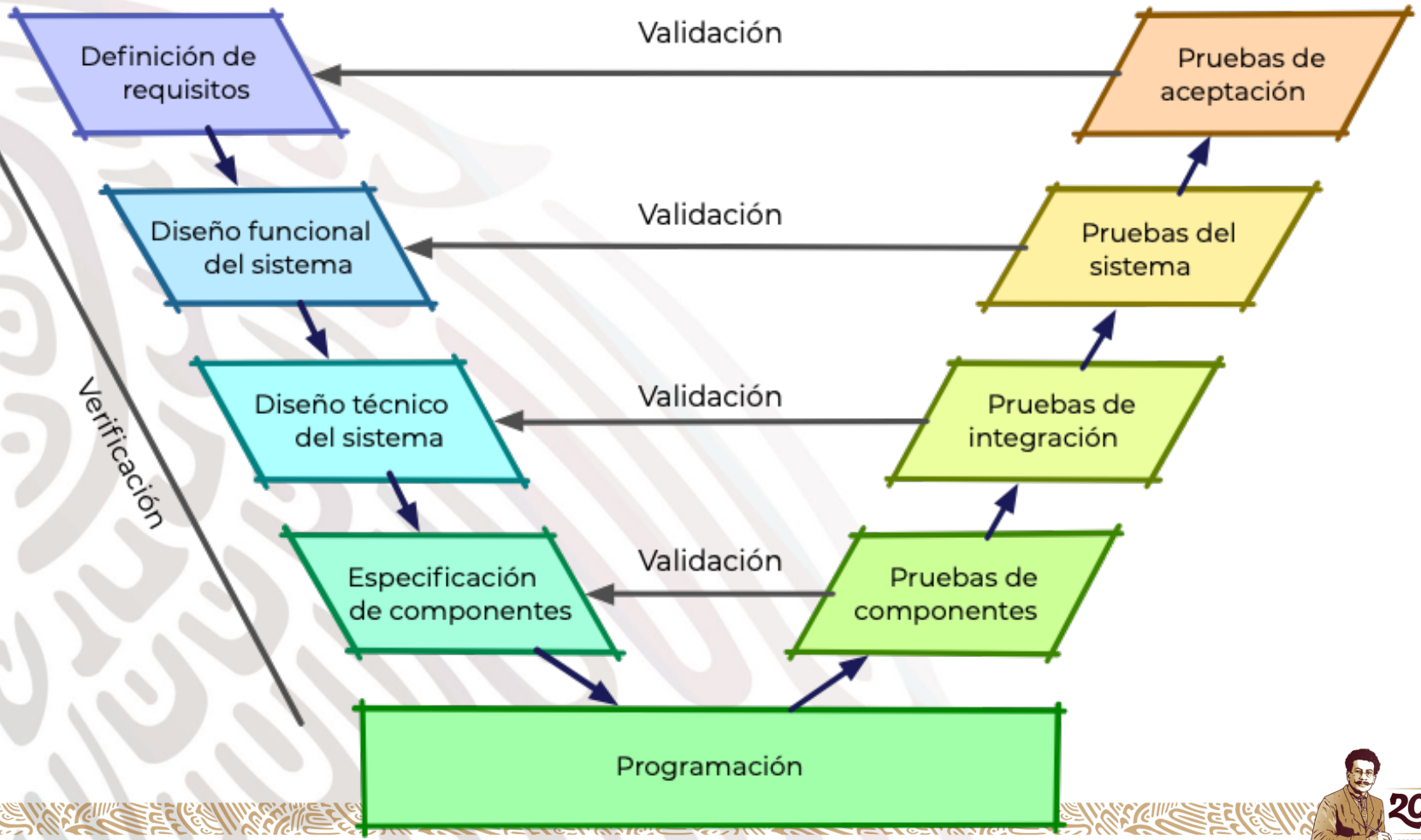


El modelo de desarrollo en V

- El modelo de desarrollo en V se parece mucho a una cascada pero colocando énfasis en las pruebas.
- Este modelo de pruebas se usa como base para explicar que es un Nivel de pruebas.
- Las pruebas de un sistema se recomiendan en todos los niveles del sistema.
- Un nivel de pruebas hace énfasis en que elementos se están probando y bajo que perspectiva.
- Los niveles ayudan también a definir los tipos, técnicas y herramientas de pruebas a utilizar.



Modelo de desarrollo en V



Explicación del modelo en V

- Definición de requerimientos Se refiere a la especificación de las necesidades del usuario que debe cubrir el sistema.
- Diseño funcional del sistema Se refiere a la especificación y diseño a detalle del sistema.
- Diseño técnico del sistema Se refiere al diseño de la arquitectura del sistema.
- Diseño de componentes Se refiere al diseño de las partes específicas a construir del sistema: clases, métodos, variables, algoritmos, etc.
- Programación Se refiere a la construcción de los artefactos de prueba del sistema.

Explicación del modelo en V (cont.)

- Pruebas de componentes En la actualidad pa programación se divide entre arios programadores y cada programador prueba independientemente sus componentes antes de liberarlos a un repositorio para su integración al proyecto final.
- Pruebas de integración Una vez integrado el código de varios componentes se pueden probar dos o más componentes juntos a fin de verificar que se comunican y colaboran adecuadamente.
- Pruebas del sistema Una vez integrados todos los componentes se ejecutan pruebas con todo el sistema junto: incluyendo servicios externos, interfaces, dispositivos, bases de datos, etc.
- Pruebas de aceptación Una vez que el equipo de desarrollo ha terminado el sistema es necesario que el usuario final verifique la funcionalidad del sistema.

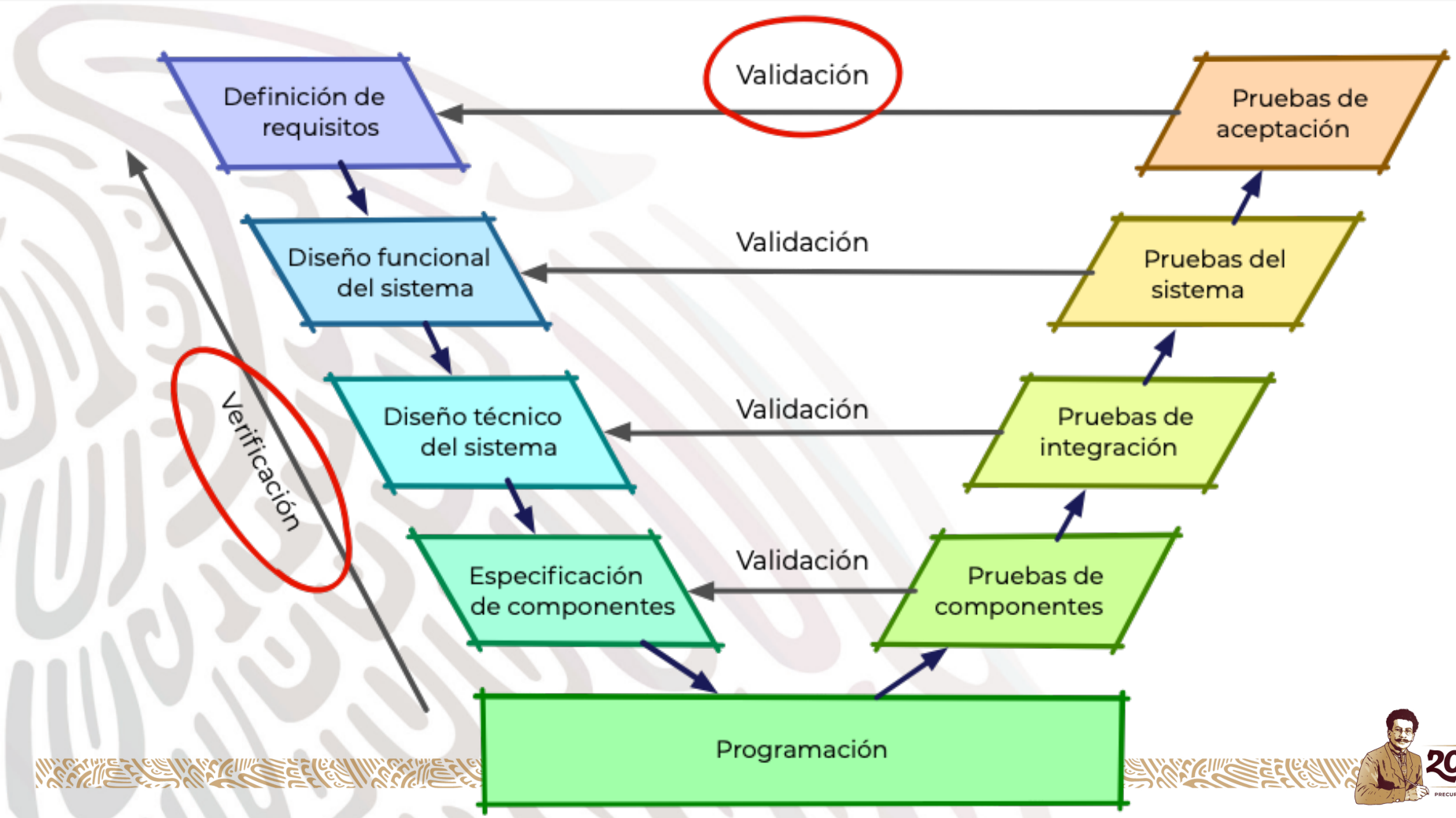


Niveles de pruebas

- Pruebas de componentes Valida que cada componente cumpla con su especificación (diseño de componente).
- Pruebas de integración Valida que los componentes colaboran adecuadamente con base en la especificación de la arquitectura.
- Pruebas del sistema Valida que el sistema cumpla con su especificación funcional.
- Pruebas de aceptación Valida que el sistema hace lo que el usuario pidió.



Verificación VS Validación



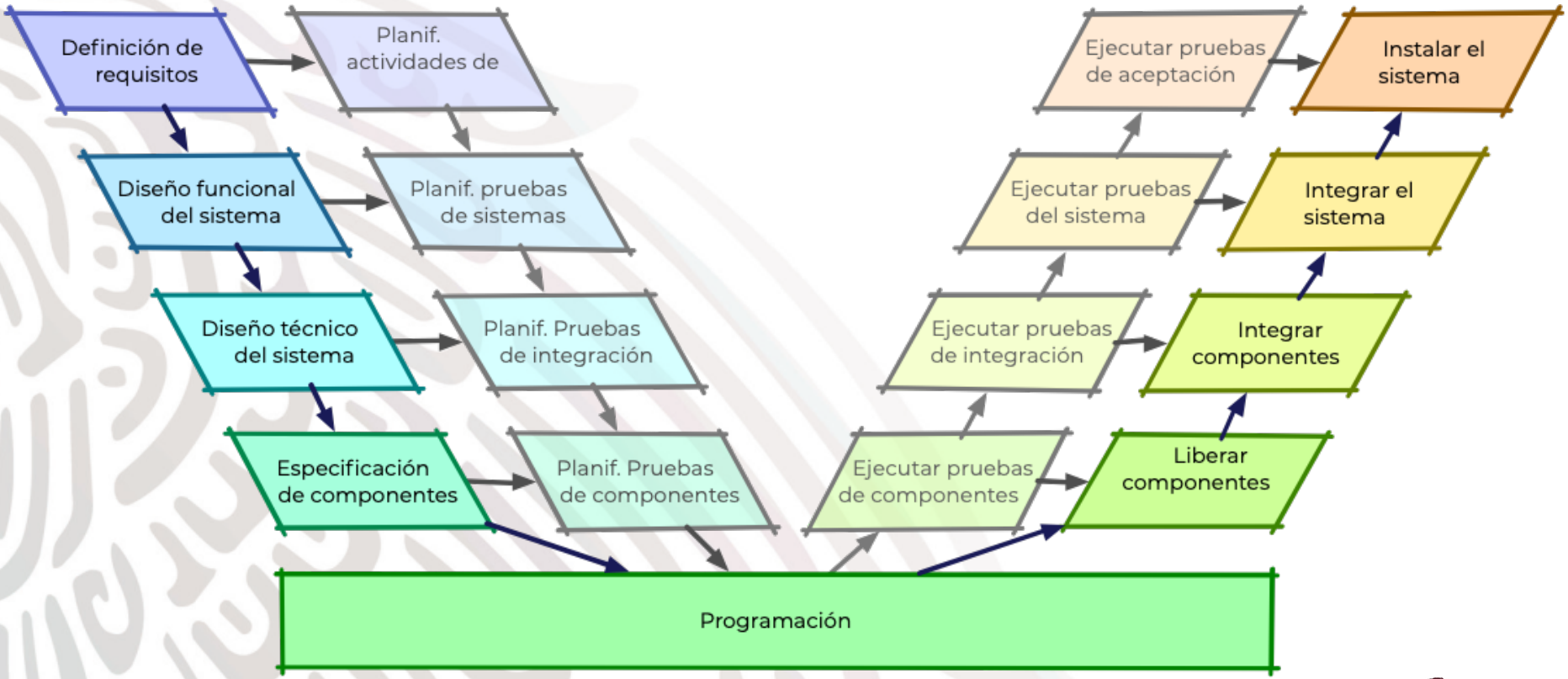
- Verificación Proceso en el que se determina si el producto cumple con una especificación. Se pone a prueba al producto construido versus su especificación (ya sea especificación de requerimientos deHace lo que acordamos?
- Validación Proceso en el que se determina su el producto construido tiene validez para el usuario, es decir, responde a la pregunta: ¿le sirve?
- Esta definición difiere un poco de lo que marca el modelo en V.



- El modelo en V propone diferentes niveles de pruebas y establece que cada nivel de pruebas es utilizado para verificar diferentes especificaciones del sistema.
- Sin embargo, da la impresión de que el proceso de pruebas comienza justo después de la etapa de programación.
- El modelo en V es una mejora a la cascada pero si se lleva a cabo de esta manera se tendrían todas las desventajas de el modelo de proceso de cascada.



Modelo de desarrollo W

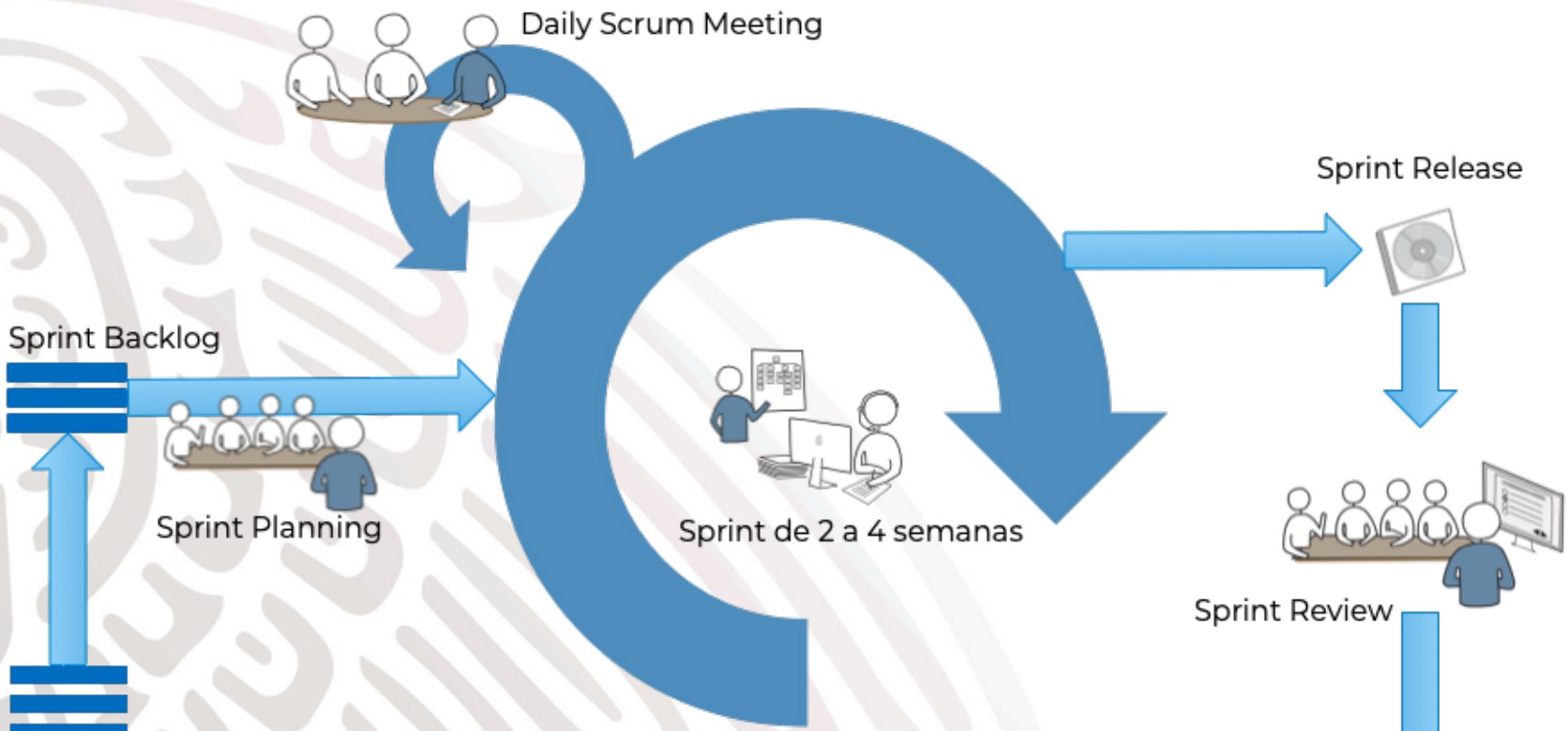


Explicación del Modelo-W

- Esta mejora o extensión al Modelo-V enfatiza que el proceso de pruebas comienza mucho antes de la programación, ya que en el momento en el que existe una especificación se inicia la planeación de las actividades de pruebas.
- Determina que cada nivel de pruebas se ataca por separado y se desarrolla en momentos determinados.
- Sin embargo, sigue teniendo los mismos problemas que el proceso de cascada.
- El Modelo-W si bien no se utiliza siempre como un proceso de desarrollo de software tal cual, sirve de base para poner en relieve las actividades del proceso de pruebas y cómo estos se relacionan con las demás actividades de desarrollo.



Modelos iterativos



Pruebas en los modelos iterativos

- Los modelos iterativos deben ejecutar pruebas en cada iteración, ya que cada incremento construye una nueva característica.
- Una de las desventajas de desarrollar iterativamente es que en cada incremento se deben repetir las Pruebas de Regresión.
- Es por esto que las metodologías ágiles recomiendan ampliamente la s Pruebas automatizadas.
- Scrum en particular tiene como actividad principal el Sprint Review el cual es una especie de prueba de aceptación.
- Para lograr que en Scrum se tenga al final de cada iteración un "producto terminado", este debió ser probado exahustivamente hasta alcanzar el criterio de aceptación, lo cual no es práctico.
- Una práctica común, no muy recomendada es la de dejar iteraciones al final exclusivas de pruebas.



1 Procesos de pruebas

2 Modelos de desarrollo

3 Niveles de prueba
- Pruebas de componentes
- Pruebas de integración
- Pruebas de sistema
- Pruebas de aceptación



Pruebas de componentes

- También conocidas como Pruebas Unitarias, tienen como finalidad probar cada componente justo después de su construcción.
- La definición de componente depende del lenguaje de programación, técnica o herramienta de desarrollo:
  - Java: Clase, Paquete o Método.
  - Base de datos: Entidad, Store procedure, etc.
  - Lenguaje C: Función, Módulo, etc.
- La ejecución de este tipo de pruebas requiere del conocimiento del lenguaje de programación, su especificación y ejecución por lo que son diseñadas y ejecutadas principalmente por los desarrolladores.
- Gracias a este último punto también se les conoce a este tipo de pruebas como Pruebas de Desarrollo.
- Solo se prueban componentes individuales (no siempre es posible y casi siempre se requieren controladores o arneses de prueba).
- Cada componente se prueba de forma independiente.



Características de las pruebas de componentes

- La Base de pruebas para diseñar estas pruebas son:
  - Requisitos del componente.
  - Diseño detallado.
  - Código.
- Los Objetos de pruebas suelen ser:
  - Clases.
  - Componentes.
  - Programas.
  - Código.
- Los Tipos de pruebas que se pueden ejecutar son:
  - Pruebas de caja blanca.
  - Pruebas basadas en la experiencia.
  - Pruebas estáticas (inspección de código, métricas de diseño).



Implicaciones de las pruebas de Componentes

- No permiten conocer factores de comunicación entre componentes ni con su entorno de ejecución.
- No permiten validar efectos cruzados o de simultaneidad.
- Son sensibles a defectos introducidos por los Test Stubs.
- No permiten probar propiedades emergentes del sistema: Tamaño, tiempo de respuesta, seguridad, etc.



Nuevos conceptos

- Arnés de prueba Código adicional que requiere un componente para ser probado independientemente. el arnés puede simular el comportamiento de otro componente dependiente, proporcionar datos, simular respuestas u otro sistema, etc.
- Driver de pruebas Biblioteca utilizada para simular las respuestas dadas por componentes aun no existentes o que no se desean probar.
- Test Stubs Cualquier elemento adicional que se deba construir para dar soporte a las pruebas de componentes, incluye a los arnéses de pruebas y los drivers de pruebas.



Pruebas de integración

- Permiten verificar que dos o más componentes (que se supone funcionan correctamente de forma independientemente) funcionan correctamente en conjunto.
- Este tipo de pruebas permite revelar defectos derivados de su comunicación, interconexión e intercambio de datos.
- En principio si funcionan correctamente cada componente de forma independientemente, el componente debe dar respuestas adecuadas bajo los mismos escenarios ya probados, sin embargo suelen fallar debido a que los Test Stubs estén ocultando o introduciendo alguna desviación al comportamiento de los componentes.
- Sin embargo, las pruebas de integración permiten probar otros escenarios no disponibles en las pruebas de componentes: Casos erroneos, valores a la frontera, condiciones combinadas, propiedades emergentes, etc.



Características de las pruebas de integración

- La Base de pruebas para diseñar estas pruebas son:
  - Requisitos del componente.
  - Diseño completo del sistema.
  - Arquitectura.
  - Flujos de trabajo.
  - Casos de uso.
- Los Objetos de pruebas suelen ser:
  - Implementaciones
  - Interfaces.
  - Protocolos.
  - Infraestructura.
- Los Tipos de pruebas que se pueden ejecutar son:
  - Pruebas de caja blanca.
  - Pruebas basadas en la experiencia.
  - Pruebas estáticas (inspección de código, métricas de diseño).

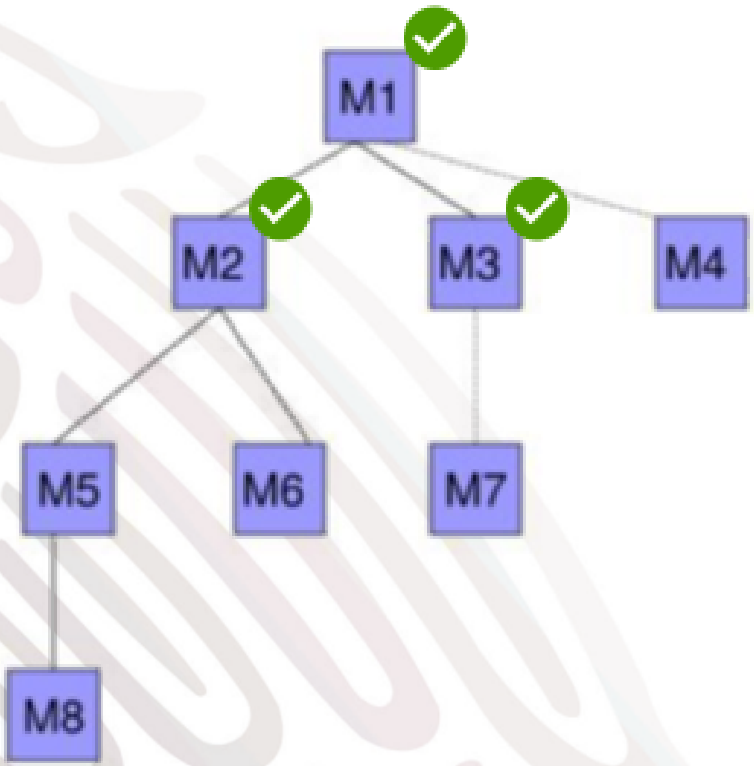


Características de las pruebas de integración

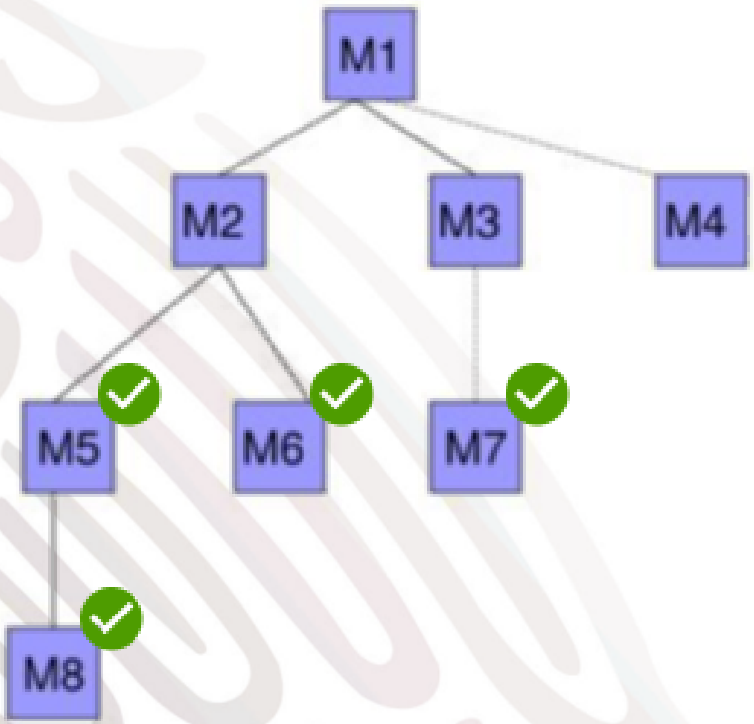
- Pueden ser ejecutadas por desarrolladores, testers o ambos.
- Aunque se suelen remover Test Stubs, algunos seguirán siendo necesarios y generalmente se reutilizan.
- Cualquier prueba que requiera ocultar o separar una parte del sistema para su ejecución es de integración.
- como prueba partes del sistema tanbien se les conoce como Pruebas de subsistemas.



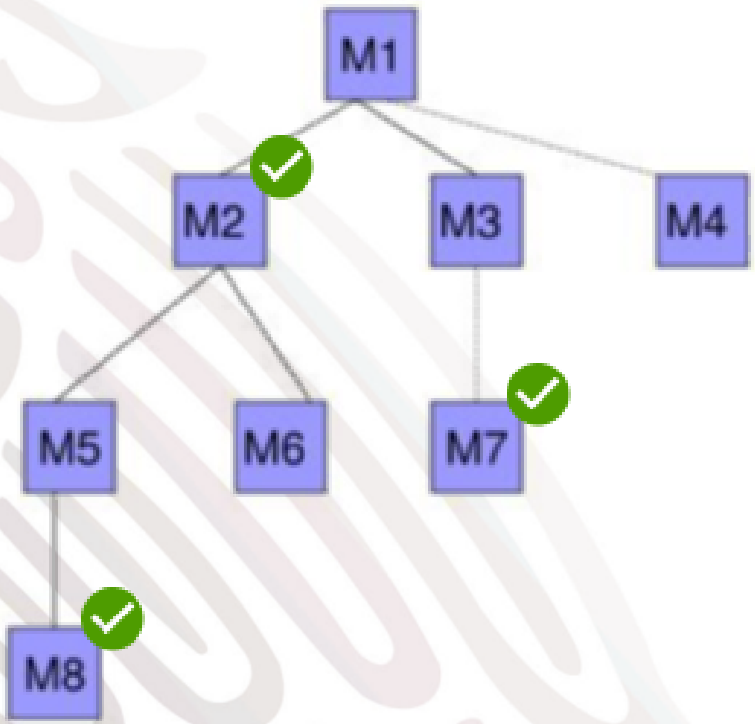
Estrategia Top-Down



Estrategia Bottom-Up



Estrategia Big-Bang o Add-Hoc



Pruebas del sistema

- Este tipo de pruebas consisten en probar el sistema completo en un entorno equivalente o sumamente parecido al entorno de producción final.
- No se requiere ningún tipo de Test Stub.
- Estas pruebas son también conocidas como Pruebas de Fábrica.
- Tiene como finalidad verificar que el sistema cumple con todos sus requerimientos.
- La calidad del sistema toma el punto de vista del Usuario.
- No se debe usar el entorno real.
- Todos los sistemas externos se configuran a condiciones reales.
- Permite probar todas las propiedades emergentes que se deseen.
- Normalmente se prueban todas las funciones y se documentan los hallazgos y evidencias de que el sistema funciona completamente.



Características de las pruebas de sistema

- La Base de pruebas para diseñar estas pruebas son:
  - Requisitos del sistema.
  - Requerimientos del usuario.
  - Casos de uso.
  - Manuales de usuario.
  - Configuración del sistema.
  - datos del sistema.
- Los Objetos de pruebas suelen ser:
  - Todo el sistema.
- Los Tipos de pruebas que se pueden ejecutar son:
  - Pruebas de caja negra
  - Pruebas estáticas (Listas de verificación de requerimientos).



Pruebas de aceptación

- Permiten validar todos los requerimientos contractuales.
- Deben servir para determinar si el objetivo del proyecto se cumplió.
- Generalmente se realizan en presencia del usuario.
- En este proceso se suele seleccionar un conjunto de casos de prueba representativos.
- Suelen realizarse utilizando el entorno del cliente.
- Estas pruebas suelen servir para revelar defectos relacionados con: datos reales, usuarios reales, infraestructura final del cliente, tareas de mantenimiento, tareas de carga, proceso de instalación realizado por el cliente, transferencia tecnológica, interacción con otros sistemas, etc.
- La aceptación del sistema suele determinar el inicio del periodo de garantía, así como la ejecución de pruebas Alfa y pruebas Beta.



Nuevos conceptos

- Pruebas Alfa Pruebas realizadas con usuarios potenciales o reales en un entorno de o bajo el control del Proveedor.
- Pruebas Beta Pruebas realizadas con usuarios potenciales o reales en el entorno del cliente.

